

Modeling Robot Control and Dynamics

Development of a Comprehensive Toolbox and User Interface

Liam Nance, Ashton Bialek-Kling, Gavin Guarino, Aaron Weber

User Manual.....	2
Example Case.....	6
1) Starting The Program.....	6
2) Making The Robot.....	6
3) Updating Mass Properties.....	7
4) Setting Initial Conditions.....	7
5) Choosing Simulation Parameters and Running Simulations.....	7
6) Calculation of Equations of Motion.....	9
Description of Code.....	10
Forward Dynamics.....	10
PD Control with Gravity Compensation.....	12
Indirect Force Control with Impedance.....	14
GUI Explanation.....	15
References.....	17

User Manual

This manual will describe the general workflow in utilizing the developed controller app.

Begin by opening the app and positioning your window so that the entire UI is visible. Upon opening it, you will have access to both the control panel and response plots, and the Robotics Toolbox will automatically initialize. First, you must select the number of joints that your desired serial manipulator has. This will populate the DH and characteristics table, as well as your initial conditions and torque tables with rows corresponding to your joint number. Then, you may fill in the DH table and characteristics table with your robot's parameters. Note that there is an additional column on the DH table, "R/P". This is to specify whether or not that joint is prismatic (0) or revolute (1). If you have initial joint positions or velocities, then you may enter them in the initial conditions table under q_0 and $\dot{q}_{dot0}$, respectively. Finally, you may select the control type and the desirable boundary conditions. There are three possible programs that may be run;

- 1) No control: The robotic arm is simulated over the simulation time by applying torques to each joint variable. This is accomplished by ensuring that the control switch is flipped off, which can be confirmed by the red lamp. Then, the torque menu will appear and you may enter the desired torques for each joint. Finally, select the simulation time and press calculate. Note that although the environment control is still available, it is functionally invisible to the robot.
- 2) Compliance Control: The robotic arm is controlled by selecting the desired end effector location. To use this, ensure the control switch is turned on, which is indicated by a lit green lamp. If on, you will have access to select either compliance or impedance. Select compliance and tune your desired gains below. Now, you may define your boundary through the shape, length, and environmental gain. The shape will always be centered at the origin and may be none, a cube, or a sphere. The "length/diameter" controls the length of one side of the cube or the diameter of the sphere. Finally, you may select the end effector position as an X, Y, Z coordinate in the base frame. Pressing calculate will begin the computations and run the compliance control scheme.
- 3) Impedance Control: Similar to compliance control, however you will select impedance instead of compliance. Follow the same steps as above and upon pressing calculate the impedance control scheme will be run.

After pressing calculate, 6 plots will appear;

i) On a separate window, a figure of the robot will be animated showing the path that the robot took in reaching the minimal error configuration. If there is an environment selected, that environment will be plotted with a red wireframe. The desired end effector position is noted by the green star.

ii) In the main window, you will see the 5 figures plotting the environment contact force, the end effector position error, and the joint positions, velocities, and accelerations.

Underneath the end effector error plot are the equations of motion for the robotic arm. Upon pressing calculate this box will be populated with the numeric indices of the B, C, and G matrices. If you desire an analytic expression, you may press “Derive EOM (sym)” to receive the five fully articulated equations of motion. Note that for high joint systems, this may be a computationally intensive process.

In a less verbose manner,

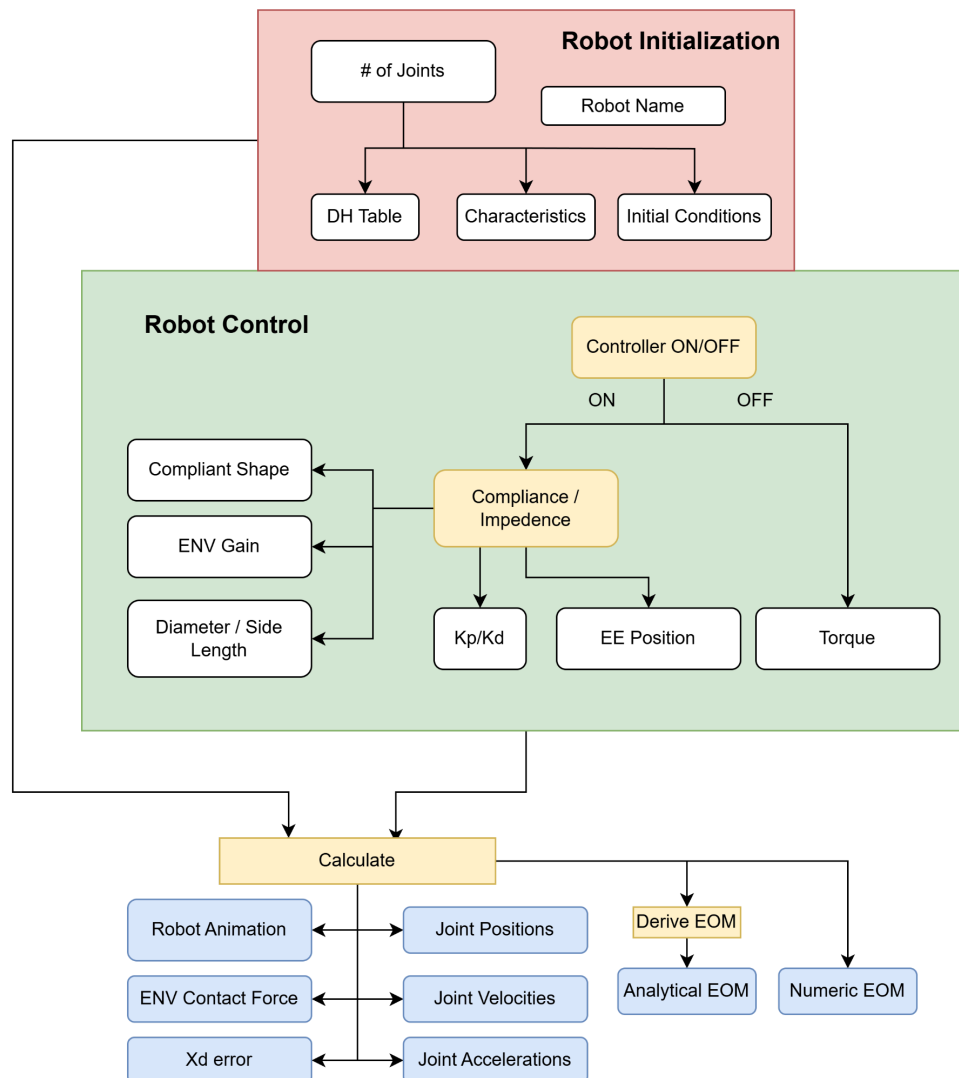
- 1) Select a number of joints and fill in DH, Characteristic, and IC tables.
- 2) Select desired control (or lack thereof) and gains.
- 3) Define environmental conditions.
- 4) Choose desired end effector position (or joint torques for uncontrolled case).
- 5) Define simulation time and calculate.
- 6) If desired, derive analytic equations of motion.

Below is the GUI with the respective steps labeled.

The screenshot shows a GUI for a robotic arm simulation. The interface is divided into several sections:

- Top Left:** A table for defining joint parameters (DH, Characteristic, and IC tables). It has columns for θ , d , a , α , and $P(0)/R(1)$. A red box labeled '1' is next to the table.
- Top Right:** Two empty plots: 'Environment Contact Force' (Force (N) vs Time (s)) and 'Joint Positions' (θ vs Time (s)).
- Middle Left:** A table for defining end effector parameters (mL, mM, ILx, ILy, ILz) and initial conditions (q0, qdot0). A red box labeled '1' is next to the table.
- Middle Right:** Two empty plots: 'EE Position Error' (Error (m) vs Time (s)) and 'Joint Velocities' (qdot vs Time (s)).
- Bottom Left:** Control and simulation settings. It includes a 'Compliance' toggle (On/Off), a 'Controller Type' (Compliance/Impedance), 'ENV Gain' (5e+04), 'Diameter / Side Len...' (3), 'Desired End Effector Position' (X, Y, Z), 'Kp' and 'Kd' gain sliders, and 'Sim time (s)' (20). A red box labeled '2' is next to the 'On' toggle, a green box labeled '3' is next to the 'ENV Gain' field, and a green box labeled '5' is next to the 'Sim time (s)' field.
- Bottom Center:** A 'Calculate' button.
- Bottom Right:** A plot for 'Joint Accelerations' (qddot vs Time (s)).
- Bottom Center-Right:** A section for 'Equations of Motion'. It contains a 'Derive EOM (sym)' button and a text box that says 'Equation of motion will display here after calculate.' A blue box labeled '6' is next to the 'Derive EOM (sym)' button.

Below shows the workflow as a flow chart for a visual understanding of the system.



Possible error messages and solutions:

Error	Solution
Error using matlab.graphics.primitive.Group/set Invalid or deleted object.	The user closed the animation prematurely. This error will not affect the performance of the code and may be safely ignored.
End-effector at initial joint angles is outside the environment boundary (penetrating). $x_0 = [2.000 \ 0.0000 \ 0.000]$. Adjust q_0 or environment size.	The end effector initial position is within the environment. This results in a large force spike and erratic behavior for the remainder of the simulation. To continue, enlarge the

	environment or reposition the end effector so it does not initialize within the boundary.
Invalid value for Matrix property.	There is a missing or NaN value in one of the tables. Double check inputs to ensure continuity with the DH table. NOTE: “pi” is not supported, you must type pi/2 as 1.578...
Index in position 2 exceeds array bounds.	No number of joints selected.

Additional Note: Depending on the robotic configuration, it may take several seconds for the animation to plot. You may view the progress of the code in the command window on MATLAB. When pressed, the following text will appear;

```

--- Calculate pressed ---
Mode: xxxx | n = x | tspan = [0 x]
      q0  = x
      qd0 = x
      xd  = [x x x]
      Kp diag = xxx | Kd diag = xxx

```

This means the app is actively running. When it is finished, the following will pop up beneath this,

Sim done.

Example Case

Anthropomorphic arm with spherical wrist

This example case will walk you through building a six-degree-of-freedom arm using the toolbox's GUI, simulating its dynamic response, and generating plots of force, position, velocity, and acceleration data.

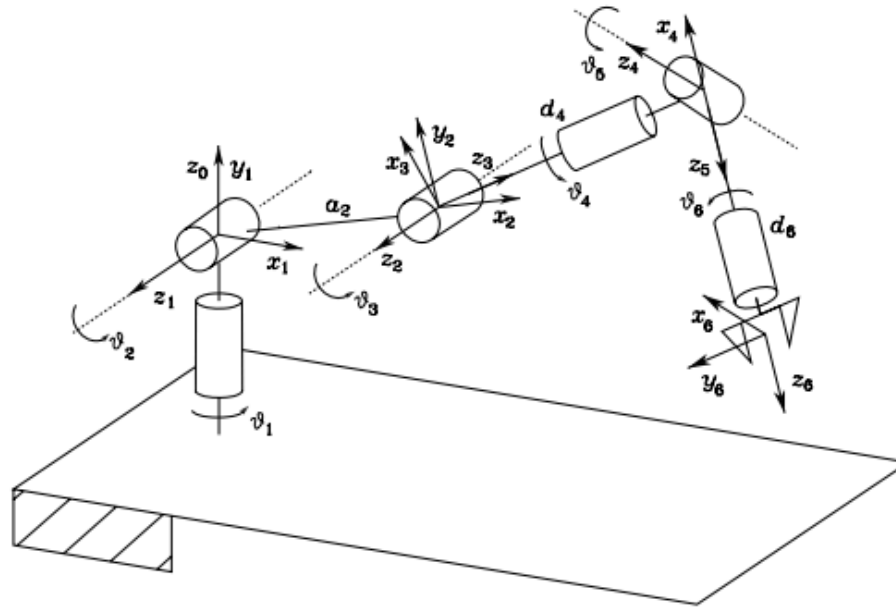


Fig. 2.26. Anthropomorphic arm with spherical wrist

Table 2.6. DH parameters for the anthropomorphic arm with spherical wrist

Link	a_i	α_i	d_i	ϑ_i
1	0	$\pi/2$	0	ϑ_1
2	a_2	0	0	ϑ_2
3	0	$\pi/2$	0	ϑ_3
4	0	$-\pi/2$	d_4	ϑ_4
5	0	$\pi/2$	0	ϑ_5
6	0	0	d_6	ϑ_6

1) Starting The Program

Open the robot toolbox GUI. Open the file named RobotGUI_Final and press run, or paste “run('RobotGUI_Final.m')” into the command window and press enter.

2) Making The Robot

Navigate to the DH Table in the top left and input the desired number of joints. In this case, use 6 joints. **Replicate the DH table above** and use the following values for a_2 , d_4 , and d_6 . The theta values can be left blank because they are the independent joint variables.

a_2	d_4	d_6
5	5	5

The joint type of each link must be specified, which is done using the final column of the DH table. Enter 0 for a prismatic joint (linear motion) and 1 for a revolute joint (rotational motion).

Enter 1 for all links for this example.

3) Updating Mass Properties

The table below the DH Table is where you can update the masses and moments of inertia of the links and motors of the robot, and the gear ratios of the motors. Replicate the table below.

Note that all links must have a non-zero positive mass. The motor masses can be set to zero if they are negligible compared to the link masses.

mL	mM	ILx	ILy	ILz	IM	kr
1	0	0	0.1	0.1	0.3	110
10	0	0	1.5	1.5	0.3	110
1	0	0	0.1	0.1	0.3	110
10	0	0	1.5	1.5	0.3	110
1	0	0	0.1	0.1	0.3	110
10	0	0	1.5	1.5	0.3	110

4) Setting Initial Conditions

The table to the right of the mass and moment of inertia table is where the initial pose and velocity of the robot are defined. For now, the choice is arbitrary, but for future use, these values will come from solving the inverse kinematics problem for a robot's initial pose.

For now, set:

$q_4 = 1.57$

5) Choosing Simulation Parameters and Running Simulations

Below the input tables will be a toggle switch and a slider that control the control schemes of the robot.

Note: There are three different ways the robot motion can be simulated. A free response with joint torques applied to the robot, and two control schemes: Compliance Control with Gravity Compensation and Indirect Force Control through Impedance Control. The

free response is useful for viewing how the robot moves, and the control schemes will make the robot move to a specified endpoint.

To turn the control schemes on, flip the toggle switch to on. A green light will indicate that the control schemes are selected.

The slider switch will switch between the two control schemes.

To view the free response of the robot, switch the toggle switch off. A red light will indicate that the control schemes are not selected.

a) Control with no environmental effects

Make sure the Compliant Shape dropdown menu is set to “None”

Enter the desired end effector x,y,z coordinates in the cells labeled “Desired End Effector Position”. **For this example, choose (10,4,8).**

The knobs allow you to change the gains of the controller, which will change how the robot responds. Keep them in their default values, but feel free to change them later.

Set the simulation time to the left of the calculate button. The unit of time is in seconds. **Anything between 5 and 10 seconds will be sufficient** for this example. Press the calculate button ONCE and wait for the robot's response to plot.

b) Control with environmental effects

Choose a boundary for the robot to operate in in the Compliant Shape dropdown menu. For this example, choose “Box”. Choose the ENV Gain, which is essentially the environmental stiffness. For this example, leave it as the default 5e4 value. **Set the Diameter/Side Length Parameter to 20.**

Enter the desired end effector x,y,z coordinates in the cells labeled “Desired End Effector Position”. **For this example, choose (15,4,8).**

The knobs allow you to change the gains of the controller, which will change how the robot responds. Keep them in their default values, but feel free to change them later.

Set the simulation time to the right of the calculate button. The unit of time is in seconds. **Anything between 5 and 10 seconds will be sufficient** for this example. Press the calculate button ONCE and wait for the robot's response to plot.

The desired position is intentionally outside of the environment, and the robot will not make it to the desired position.

c) Free Response

With the toggle switch turned off, the panel to the right of the knobs will allow the input of joint torques and forces to each motor on the robot. Make sure the compliant shape is set to “None”

For this example, set the **torque for joint 2 to 50 Nm**, and the **torque for joint 5 to 6 Nm**. Set the simulation time to the right of the calculate button. The unit of

time is in seconds. **Anything between 10 and 15 seconds** will be sufficient for this example. Press the calculate button ONCE and wait for the robot's response to plot.

6) Calculation of Equations of Motion

The numeric equations of motion for the robot in the joint space will be calculated automatically with any simulation. To calculate them analytically and without running a simulation, press the button labeled “Derive EOM (sym)”.

Things to note: Angle inputs must be in radians

Resize the figure to unlock the 3d robot plot and move it around

Close the 3d plot window before each simulation to remove old target points and boundaries

Description of Code

Forward Dynamics

The response of the robot due to generalized input forces and no control loops is handled by the function runFWDdyn. The forward dynamics problem is solved by solving the robot equations of motion to find the values of the joint variables, joint variable velocity, and joint variable acceleration for a set amount of time. Once the values of the joint variables are found, they can be plugged into the forward kinematics equation to find the position and orientation of the end effector with respect to the base frame. The equations of motion are solved using MATLAB's ode45 combined with a function to update the equations of motion with the current joint variables at each time step.

The equations of motion are found by finding three key matrices: B, C, and G, which represent mass and inertia effects, Coriolis and centrifugal forces, and forces due to gravity, respectively. Three functions are foundational to the derivation of the equations of motion, both symbolically and numerically. To calculate the symbolic equations of motion, the toolbox uses the functions makeBmatrix, makeCmatrix, and makeGravitymatrix. This toolbox requires the installation of MATLAB's Symbolic Math Toolbox to make the symbolic calculations. To calculate the numeric symbolic equations of motion, the toolbox uses the function makeBmatrix, makeGravitymatrix, and the SerialLink.coriolis function from Corke Robotics Toolbox.

The makeBmatrix function takes the inputs of the robot DH table, a list of the joint types, the link masses, the motor masses, the link moments of inertia, the motor moments of inertia, the gear ratios of the motor, and the requested solution type (numerical or symbolic). The function makes calls to two other functions: findTransformMatrix and linkMotorJacobian. The B matrix function extracts the rotation matrix from the homogeneous transformation matrix that has been calculated from each link to the base frame. The function linkMotorJacobian is called to provide the jacobians of each link and motor used to relate the linear and angular velocities to the joint variables. The B matrix is calculated by summing B_i terms which are found using the following equation.

$$B_i = m_{li} * J_P^{(li)T} J_P^{(li)} + J_O^{(li)T} R_{i li}^i I_i^T J_O^{(li)} + m_{mi} J_P^{(mi)T} J_P^{(mi)} R_{mi mi}^{mi} I_{mi}^T J_O^{(mi)}$$

The J terms represent the various Jacobians, the R terms are the rotation matrices for each link, and the I terms are the moment of inertia tensors for the links and the motors. The rotation matrices are extracted from the homogeneous transformation matrices calculated from the base frame to each link. Finally, the m terms represent the masses of each link and motor.

The makeCmatrix function only outputs a symbolic matrix since the formula for calculating it involves derivatives. Each element of the C matrix is a sum of the Christoffel symbols of the first type. The function is structured as a triple for-loop that iterates through the links and each element of the B matrix. The elements of C are calculated using the following equation.

$$C_{ij} = \Sigma \frac{1}{2} \left(\frac{\delta b_{ij}}{\delta q_k} + \frac{\delta b_{ik}}{\delta q_j} - \frac{\delta b_{jk}}{\delta q_i} \right) \dot{q}_k$$

The gravity matrix represents the potential energy of the system and is an nx1 matrix. Each element of the matrix is calculated using the following equation.

$$g_i = - \Sigma (m_{lj} g_0^T P_{Pi}^{(lj)} + m_{mj} g_0^T J_{Pi}^{mj})$$

The function makeGmatrix loops through all the links for each element of g, and then for each element of g, loops through the links again to find each term in the sum.

The function linkMotorJacobian is an integral function for the rest of the forward dynamics functions since every formula for calculating them depends on the jacobians. The function takes inputs of an array of homogeneous transformation matrices for each link, the joint types, the number of links in the robot, the gear ratios of the motors, and the requested solution type. The function is structured by a nested for loop. The first one iterates through the links to find the Jacobian matrices for each one, and the inner loop iterates up to the local link number to find the Jacobians for each column. The function will read the jointType vector to determine whether the jacobians will be calculated based on whether the joint is prismatic or revolute. The function makes a couple of key assumptions that could be changed in the future, but would increase the number of inputs the user would need to provide. This function assumes that the links are homogeneous, so the center of gravity will be at the center of the links. The function also assumes that all motors lie at the base of each link they actuate. This assumption makes sense for a robot with only revolute joints; however, the user could encounter issues when modeling a robot with prismatic joints. In some prismatic joints, the motor could move with the frame it is actuating, so the dynamic response of the system would be different than what is currently modeled if that is the case. Each column of the Jacobian matrix is calculated using the following equations. J represents the index of the local link/frame, and i represents the index of each link in the robot. Within the function, vector arrays are made to store the position of each frame and the direction cosines of each z-axis with the base frame occupying the first index. That way, the j index in the code is equivalent to the (j-1) index in the equations.

$J_{Pj}^{(li)}$	z_{j-1}	prismatic
$J_{Pj}^{(li)}$	$z_{j-1} \times (p_{li} - p_{j-1})$	revolute
$J_{Oj}^{(li)}$	0	prismatic
$J_{Oj}^{(li)}$	z_{j-1}	revolute
$J_{Pj}^{(mi)}$	z_{j-1}	prismatic

$J_{pj}^{(mi)}$	$z_{j-1} \times (p_{mi} - p_{j-1})$	revolute
$J_{oj}^{(mi)}$	$J_{oj}^{(li)}$	<i>for j = 1, ... , i - 1</i>
$J_{oj}^{(mi)}$	$k_{ri} z_{mi}$	<i>for j = 1</i>

The final function utilized by the dynamics functions is findTransformMatrix. The function outputs the homogeneous transformation matrix from the base frame to the end effector, and a collection of homogeneous transformation matrices from the base to the frame of all different links in the robot. The function simply builds the transformation matrix for each row of the DH table supplied to the function, multiplies them, and stores those matrices for each link.

PD Control with Gravity Compensation

The runComplianceSim script makes use of the functions written above to calculate the required joint torques to move the end effector to a specified location, $x_desired$. Compliance control works by treating the robot arm assembly as a generalized damped spring. For typical forward dynamics, the joint positions, velocities, and accelerations can be computed as a function of time by inputting the torques in each joint. However, for compliance control, the joint torques are calculated using the relationship:

$$\tau = [J^T K_P (x_{des} - x_{act}) - J^T K_D J (\frac{d}{dt} q)] + G + J^T F_{env}$$

Where:

- τ = Required joint torques for compliance control
- J^T = Jacobian transpose for series arm
- K_P = Arm proportional gain (equivalent spring stiffness)
- x_{des} = Desired end effector location
- x_{act} = Actual end effector location
- K_D = Arm derivative gain (equivalent damping)
- $\frac{d}{dt} q$ = Joint velocity vector
- J = Jacobian for series arm
- F_{env} = Force exerted by the environment
- G = Force due to gravity (gravity matrix)

The first term is the PD controller (standing for Proportional, Derivative), and prescribes how the series arm acts as a compliant mechanism. The “G” relates the joint torques to the force of gravity and ensures that it is cancelled out when the forward dynamics are computed.

Cancelling gravitational effects allows the arm to move without sagging due to gravity. The last term is the force on the arm due to environmental interactions. When the arm contacts a simulated surface, the surface pushes back on the arm using Hooke's law. The force due to the environment is modeled as:

$$F_{env} = -K_{env}(x_{act} - x_{bound})$$

Where:

F_{env} = Force exerted by the environment

K_{env} = Environment proportional gain (wall equivalent stiffness)

x_{act} = Actual end effector location

x_{bound} = Location of the undeformed wall

So, the wall is essentially treated as a spring with equivalent stiffness, K_{env} , that responds with a force proportional to Δx . This means that it can deform slightly, as real materials do, and settle in the location where the spring force from the robotic arm equalizes the spring force of the wall. This force is plotted against time to visualize how much force the robot arm applies to the wall, since, due to the laws of motion, the robot and wall must push equally against each other. The force that is plotted should oscillate before settling to a final force, since the algorithms are akin to damped springs (meaning the oscillation cannot continue infinitely).

The objects coded into this program are "box" and "sphere." The "box" object creates a 3d cube around the robot, and the "sphere" creates a 3d sphere around the robot. These objects are simulated using the force algorithm outlined above, enforced at the user-defined boundary. The function F_{env} is responsible for computing these forces, while the $drawBox$ and $drawSphere$ functions create wireframe representations of the bounds. Note that due to the force scheme, the arm must be initialized within the boundary, because starting outside the bounds will create an unreasonably large force. To prevent this from happening unintentionally (since the user only inputs joint positions and may not know where the end effector will be positioned), the script will output an error if the end effector is initialized outside of the bounds.

The gains K_p , K_D , and K_{env} can be tuned to simulate real-world behavior. To get a spring that is responsive and aggressively adjusts, K_p may be increased. However, as with all proportional controllers, this may cause the system to overshoot and oscillate around the desired endpoint. In response, the user may want to damp the system by increasing K_D . Increasing the derivative gain may make the system more stable, but the prime drawback of using this type of controller is that it makes the system substantially more sluggish. Overdamping can make the arm appear as if it is moving through a very viscous fluid. Tuning the environment stiffness will allow the wall to have as much, or as little, give as the user prefers.

Compliance control is a unique approach to robot control because it enables the robot to interact safely with the environment, rather than having to follow a rigid path that may make it prone to crashing. This prevents the robot from damaging itself and objects around it, while using a control scheme that can move the arm to a desired position without having to solve the inverse kinematics, since this code computes everything in the task space using Cartesian coordinates.

The code itself is similar to just doing basic forward dynamics. It takes DH parameters, joint and link parameters, and initial joint parameters before solving the forward dynamics using ode45. The primary difference when using PD compliance control is that instead of using torques as a user input, they are computed based on the structure above. It calculates how far the robot is from where it needs to be, and adjusts accordingly every loop. This is a closed-loop feedback controller because it constantly measures the robot's current position (q) and velocity ($\dot{q}$) to calculate the relative position error, which allows the calculation of the joint torques. The calculated torques induce acceleration on the system using the derived dynamics, which changes q and $\dot{q}$; the process repeats until the end of the time frame.

The controller's only goal is to minimize the error of the end effector position from the desired point, so if the desired point is placed outside of the bounds, the controller will make the arm press against the wall until it applies the amount of force that would be required to reach the position, while the wall pushes back to keep it within bounds.

The primary downside of compliance control is that it is not optimized for stability and performance. The arm is compliant by nature, so if the gains are not tuned properly for the situation, it can accelerate unreasonably fast, oscillate wildly, and apply too much force to the environment. Or, it may respond too slowly and supply insufficient force. The control of the arm is only as good as the gain tuning. Also, since the algorithm is only responsive to the end effector error, it will not follow a designated path.

Indirect Force Control with Impedance

Impedance control controls how a robot responds to force rather than rigidly following a positioning trajectory. This will cover the case that where an arm or any robot wants to arrive at a position but experiences any sort of force, whether it be a push from an outward device or contact with another surface, the robot will account for this to still arrive at the same point. The main idea is to make the end-effector behave like a spring-damper system where it will react for forces put onto it but when those forces are removed it will then revert to the original required forces to keep it at that location without the external forces. Rather than requiring exact joint positions, the controller shapes the robots mechanical impedance so that contact with external factors are smooth and controllable.

$$\tau = J^T [-K^d(x_{ac} - x_e^d) - K^D \dot{x}] + G + J^T F^{env}$$

Where:

τ = Required joint torques for compliance control

J^T = Jacobian transpose for series arm

K_p = Arm stiffness gain matrix

x_{ac} = Actual end effector location

x_e^d = Desired end effector location

K_D = Arm damping gain matrix

$\dot{x}$ = Cartesian end effector velocity

F_{env} = Force exerted by the environment

G = Force due to gravity (gravity matrix)

The wall is treated as a spring with equivalent stiffness K^{env} that responds with a force proportional to penetration depth. This means it can deform slightly, as real materials do, and the arm settles at the location where the impedance spring force equalizes the wall spring force. This force is plotted against time to visualize how much force the robot arm applies to the wall. Due to Newton's third law, the robot and wall push equally against each other. The plotted force should oscillate before settling to a final steady value, since the algorithm is analogous to a damped spring system.

The function `runImpedanceSim` is a function that calls for the implementation of impedance control on the robotic arm. This is the one that is called by the GUI and runs the code. It assembles a `SerialLink` model from the input DH table and joint types, verifies that it is a valid configuration and then uses `ode45` to implement. After integrating it, compute the joint accelerations and the full environment force based on the given constraints through the GUI.

This function calls on `ode45` to solve for the derivation of the joint variables at each time step for the systems. It calculates the forward kinematics of the system using the transformation matrix and Jacobian as described in previous sections. Then, the desired 6 DOF are assembled from the impedance law. The stiffness term resorts the end-effector to the desired positions while the damping term will dissipate kinetic energy. The function also goes through the forcing through the environment by simulating the environmental force through the description of the operational space. This term is added to the force developed by the controller so that the contact force is accounted for when calculating what the actual force of the end effector has to be. Next, the model is mapped to the joint forces using the Jacobian transpose and the robot is integrated using the forward dynamics using the actual inertia for the forcing terms on the end effector. This leads to an end force equation that is driven by the impedance of the end effector.

The primary downside of impedance control, like compliance control, is that performance is highly dependent on gain tuning. Poorly tuned gains can cause the arm to accelerate unreasonably fast, oscillate, or apply excessive force to the environment. The controller only minimizes end-effector position error and will not follow a designated path between start and target. However, the impedance formulation is generally more physically intuitive to tune than pure compliance control, since the stiffness and damping parameters have direct analogues to real spring-damper systems.

GUI Explanation

The GUI is intended to resemble a control panel as best as possible, grouping the user inputs to one side and displaying the relevant graphs on another. The app was initially designed within the MATLAB app designer, but then exported to a .m file to accommodate all installations of MATLAB. The following discusses the logic behind the major functions in the file.

`function` `applyControlVisibility(app)`

- We want to cycle the visibility of the torque and EE input values depending on whether or not the control switch is activated. This establishes the baseline for that.

function resizeAllTables(app, n)

- As the number of joints increase, the DH, characteristics, IC, and torque tables must grow to accommodate them. This is the framework for that functionality.

function displayEOM(app, B, C, G, n, q0, qdot0)

- Outputs the EOM in a readable format for the user. This is either printed symbolically or numerically depending on the input.

function CalculateButtonPushed(app, ~)

- Major component of the code. This activates when the calculate button is pressed. It reads the DH, Characteristics, and IC Tables and stores all values as future inputs for the control functions. It also reads the environment variables and gains. Then, it reads which control mode is activated. If the control is turned off, it goes to open loop torque control, runTorqueSim. If the control mode is turned on, and compliance is selected, it runs runComplianceSim. If control is turned on and impedance is selected, then it runs the impedance control, runImpedanceSim. These functions output the joint variables, the force history, and R, which are used in the plots and animation. It then pushes those plots to their respective figures.

function DeriveEOMButtonPushed(app, ~)

- For complicated robotic arm configurations, calculating the analytic equations of motion may be time consuming. For quality of life, the symbolic calculation is tied to the “Derive EOM (sym)” button. This allows the user to troubleshoot their inputs without being forced to perform the computationally intensive derivation every run. To derive the EOMs symbolically, the entire DH table and Characteristics are converted to symbolic variables, which are then passed through the make B,C, and G matrix functions which support symbolic derivation. Then, they are expanded into the n equations based on the number of joints.

function createComponents(app)

- Application method to position and name all components.

function fig = animateRobotWithEnv(R, q, xd, EnvType, EnvBounds)

- Assembles the environment around the robot within the figure and normalizes the view window to the workspace of the robotic arm.

References

- 1) Robotics: Modeling, Planning, and Control, Bruno Sciciliano, Sciavicco, Villani, Oriolo
- 2) Corke Robotics Toolbox
- 3) Writing Apps in MATLAB, David Garrison, MathWorks
https://www.mathworks.com/company/technical-articles/writing-apps-in-matlab.html?s_tid=srchtitle_techarticles_main_1_app
- 4) Develop Custom UI Components Programmatically
https://www.mathworks.com/help/matlab/creating_guis/develop-classes-of-ui-component-objects.html
- 5) Display Graphics in App Designer
https://www.mathworks.com/help/matlab/creating_guis/graphics-support-in-app-designer.html